

Track B · Vibe coding: build the same app by describing it

Session 4 · BDS M1 · Era 3 · ~20 minutes · work in pairs

Track A took you six steps and seventy lines of Python. This track takes one paragraph of English. You will build an app of the same shape — pick some series, pick a window, compare them, take the data home — and then judge which version you would actually put your name on.

Nothing to install. A Google account is all you need.

The one rule: describe the **outcome**, never the implementation. Do not say React, do not say database, do not say caching. The moment you specify *how*, you are back in Track A with extra steps. (There is one deliberate exception in Step 1. It is flagged when you get there, and the reason matters.)

Step 0 · Open the Build tab (1 min)

Go to aistudio.google.com → sign in with a Google account → click **Build** in the left sidebar.

You get a prompt box and, once you submit, a live preview on the right. That is the entire development environment.

Step 1 · One paragraph, one app

Paste this in and hit enter:

Build a web app that compares currencies over time. Let me pick two or more currencies against the euro, pick a start and end date, and see them on one line chart, rebased so that every line starts at 100 on my start date. Show me which currency gained the most over the window I picked. Use the Frankfurter exchange-rate service at <https://api.frankfurter.dev> — it is free and needs no API key. Fetch the rates on the server, not in the browser: the page should ask my own app for the data, and my app asks Frankfurter. Cache the result on the server, so that ten people opening the page do not cause ten fetches. Organise the project into two top-level folders: `backend/` for everything that runs on the server, and `frontend/` for everything that runs in the browser. Let me download the data I am looking at as a CSV.

Watch the preview build itself. It takes a minute or two.

Why this app: it is Track A's app in a different costume — same choices, same rebasing, same download. The data source changed because it had to: a browser app has no pandas and no bundled dataset, so it needs a live URL.

About the exception. Those two sentences about the server break the rule at the top of this sheet, and that is the point. You are not choosing a technology, you are naming two outcomes the agent cannot deliver any other way:

- **One fetch serves everybody.** That is Track A's `@st.cache_data`, moved one layer down. Without it, every visitor hammers a free service that is doing you a favour.
- **A key would have somewhere to live.** Frankfurter needs none, but the moment you swap in a paid data source, anything in the browser is readable by anyone. "On the server" is where secrets can exist at all.

That is the honest version of the rule: describe outcomes, and specify structure only when an outcome you actually care about depends on it. "Use React" is a preference. "Don't put my key in the browser" is a requirement.

The sentence about folders is a third kind, and worth being honest about too: it changes nothing at all about how the app behaves. It is there purely so that *you* can see the two halves in the file explorer without opening a single file. Code is read far more often than it is written, and asking for it to be readable is never wasted breath.

The point: read the prompt again and notice what is *still* missing. No framework, no chart library, no database. You named the destination and two constraints, and left the road to the agent.

Step 2 · Find both halves

Before you ask for anything else, click the **Code** tab in the preview pane. Because you asked for the fetching to happen on the server, you did not get one file — you got an app with two sides.

You do not need to understand all of it. Find three things:

1. **The frontend** — the files that draw the chart and the date pickers. They run on the visitor's computer, inside their browser.
2. **The backend** — the file that defines a route, something like `/api/rates`. It runs on Google's machine. Nobody can see inside it.
3. **The join** — the exact URL string the frontend fetches. It should be your own route. If it says `api.frankfurter.dev`, there is no backend. That string is the whole contract between the two halves.

Then draw this on paper and label it:

```
[ browser ] → [ your server ] → [ Frankfurter ]
```

What you are looking at

Your file names will differ — the agent invents them — but every build has these jobs, and each one belongs to exactly one half:

TYPICAL FILE	HALF	ITS JOB	TRACK A EQUIVALENT
<code>server.ts</code>	backend	Defines the route, calls Frankfurter, caches the answer	<code>@st.cache_data</code> + <code>load_data()</code>
<code>index.html</code>	frontend	The page the browser downloads — nearly empty	—
<code>main.tsx</code>	frontend	Starts the app inside that page	—
<code>App.tsx</code>	frontend	Holds the state, asks the backend for data, arranges the rest	your whole <code>app.py</code>
<code>components/CurrencySelector</code>	frontend	The picker	<code>st.multiselect</code>
<code>components/DateRangeSelector</code>	frontend	The dates	<code>st.slider</code>
<code>components/ComparisonChart</code>	frontend	The chart	<code>st.plotly_chart</code>
<code>components/WinnerCard</code>	frontend	The headline number	<code>st.metric</code>
<code>components/LeaderboardTable</code>	frontend	The table	<code>st.dataframe</code>
<code>utils/csvExport</code>	frontend	Builds the download	<code>st.download_button</code>
<code>types.ts</code>	both	The shape of the data each half promises the other	—
<code>package.json</code>	project	The library list	<code>requirements.txt</code>
<code>bun.lock</code>	project	Exact versions installed	what <code>pip freeze</code> writes
<code>.env.example</code>	project	Where a key would go — safe <i>only</i> because a backend now exists	<code>secrets.toml</code>

Two things worth pausing on. `types.ts` belongs to both halves, because a contract only works if both sides hold the same copy. And Streamlit collapsed that entire left column into six function calls — which is the clearest measure of what a framework does for you, and of what it hides.

If you got one flat pile of files instead, the agent skipped the folder instruction. Ask again:

Reorganise the project into two top-level folders: `backend/` for the server and everything that runs on it, and `frontend/` for the page, the components and everything that runs in the browser. Keep the shared type definitions where both can reach them. Do not change any behaviour.

Four questions to answer out loud with your pair:

- Which box draws the chart?
- Which box does the caching you asked for?

- If you swapped in a paid data source tomorrow, which box would hold the key?
- Which box can any visitor read the entire source code of, just by pressing F12?

Prove the split is real. Press **F12** → **Network** tab → reload the preview. You should see a request to your own `/api/...` route. If instead you see a request going straight to `api.frankfurter.dev`, the agent ignored you, everything is running in the browser, and there is no backend at all. Say exactly that in the chat and ask again.

The point: this is slide 7's Era 1 list — a frontend, a backend, an API — except nobody typed it. The layers did not go away when the typing did. They are in every web app ever built; the only thing that changes between eras is who writes them and whether you can see them. This is also the tab most people never open, and everything you skip here is something you will not be able to fix later except by asking again and hoping.

Step 3 • One feature at a time

Back in the chat. Ask for **one** thing, wait, check the preview, then ask for the next:

Add a button that pins the current view — the currencies, the date range, and which one gained most — to a table below the chart, so I can compare two different windows side by side.

Then, separately:

Add the number of days in the selected window above the chart.

Try it: ask for three changes in one message instead. It usually gets two right and quietly drops the third — and you will not notice which.

The point: this is Track A's pin button, the one that needed `st.session_state`. Here you never learn that the concept exists. Convenient today; a problem the first time it misbehaves.

Step 4 • Point instead of describing

Use **annotation mode**: highlight a part of the running app and describe the change there, rather than writing "the third box from the left, above the chart".

Try it on the chart: highlight it and ask for thicker lines and the legend above rather than beside the chart.

The point: describing a location in words is slow and ambiguous. Pointing is neither. This is genuinely better than what Track A offers — there is no "click the thing you mean" in a Streamlit script.

Step 5 • Now check whether it is correct

It looks finished. That is not the same thing. Spend five minutes here — this step is the actual point of the exercise.

1. **Verify one number by hand.** Open `https://api.frankfurter.dev/v2/rates?base=eur` in a browser tab — that is the raw source, today's rates, no app in between. Pick a currency and compare it with what your app shows. Do they agree?
2. **Check the data is live.** Change the date range to something odd — a single week, or a range ending today. If the numbers do not move sensibly, the "API call" may be decoration over hardcoded values.
3. **Check the rebasing.** Every line should start at exactly 100 on your start date. If one starts at 97, the app is rebasing to the wrong row and every comparison you make with it is wrong.
4. **Break it on purpose.** Set the end date before the start date. Ask for a currency that does not exist. A real app tells you what went wrong; a demo shows a blank screen or a stack trace.
5. **Ask the backend directly.** In the Network tab, copy your own `/api/...` request URL and open it in a new browser tab. You get the raw JSON your server produces, with no frontend in the way. Now you can tell *which half* is wrong: if the numbers are already wrong here, it is the backend's maths; if they are right here but wrong on the chart, it is the frontend's display.

Whatever you find, write it down. You will need it for the comparison.

The point: in Track A, a wrong number meant a line you could read, find and fix. Here, the fix is asking again and hoping. "It works" and "it is correct" are different claims, and only one of them has been tested.

Step 6 · Publish it

1. **Publish** (top right).
2. **Get Started.**
3. **Publish App.**

You get a live Cloud Run URL, and you can claim a custom subdomain like `your-app.ai.studio` — first come, first served.

Free tier, honestly: two published apps per account, no Google Cloud project and no credit card. One catch worth knowing before the room tries it — **if your Google account has ever had a Cloud billing account attached, you are not eligible for the starter tier**, and you will be asked to link a project instead. Use a personal account if your university one trips this.

Then compare — this is the assessment

You have now built the same idea twice. Answer these as a pair, in writing:

1. **Which was faster to a first working version?** Be honest about how long Track A's setup took, and how long Step 5 took here.
2. **Which one could you debug at 9am tomorrow** when a stakeholder says the numbers look wrong? Point at what you would actually do in each case.
3. **What did Step 5 find?** If it found nothing, did you really look, or did you trust the interface?

4. **Which would you put your name on** in a report to your manager — and does your answer change if the app is going to be used every week rather than once?
5. **Where were the two layers in Track A?** Your Streamlit script has no `/api/` route and no frontend file, yet it clearly runs in a browser. So where does each line of that Python actually execute — and did the framework simplify the split, or just hide it from you?

There is no right answer. There is a wrong one, which is assuming the newer tool is the better tool because it is newer.

When re-prompting stops working

Debugging here is a conversation, not a stack trace. When it stalls:

SYMPTOM	WHAT TO TRY
It "fixes" the bug and breaks something else	Ask for one change per message, and say what must stay the same
It insists the bug is fixed when it is not	Describe the symptom precisely — what you clicked, what you expected, what appeared
It keeps missing the same thing	Open the Code tab, find the relevant part, and quote it back in your message
Everything is tangled	Start a fresh chat with a clean prompt. Cheaper than untangling
You want to keep what you have	Export the code as a ZIP, or sync the project to GitHub

The pattern worth noticing: every one of these fixes works better the more you understand the code. The 2025 study on your reading list found exactly this — expertise does not disappear in vibe coding, it moves from writing lines to judging what a model just wrote.

If you would rather build your own idea

Go ahead — the steps do not change. Two rules keep it from going sideways in twenty minutes:

- **Name your data explicitly.** "Use the Frankfurter API" works. "Use some financial data" produces invented numbers that look real. Slide 18 of the deck has four free, no-key APIs to point it at.
- **Keep the server-side sentence.** Whatever your idea, say that the outside service is called from the server and not the browser. It costs you one line and it is what gives you two layers to look at in Step 2.
- **Keep it one screen.** Sign-ups, accounts and saved user data turn a ten-minute demo into a forty-minute one, and drag in exactly the security questions the March 2025 story on the deck is about.

Appendix · the prompts, ready to paste

Build:

Build a web app that compares currencies over time. Let me pick two or more currencies against the euro, pick a start and end date, and see them on one line chart, rebased so that every line starts at 100 on my start date. Show me which currency gained the most over the window I picked. Use the Frankfurter exchange-rate service at <https://api.frankfurter.dev> — it is free and needs no API key. Fetch the rates on the server, not in the browser: the page should ask my own app for the data, and my app asks Frankfurter. Cache the result on the server, so that ten people opening the page do not cause ten fetches. Organise the project into two top-level folders: `backend/` for everything that runs on the server, and `frontend/` for everything that runs in the browser. Let me download the data I am looking at as a CSV.

If it built everything in the browser anyway:

The page is calling api.frankfurter.dev directly from the browser. Move that call to the server: give the app its own endpoint that the page requests, and let only the server talk to Frankfurter.

If the files came out in one flat pile:

Reorganise the project into two top-level folders: `backend/` for the server and everything that runs on it, and `frontend/` for the page, the components and everything that runs in the browser. Keep the shared type definitions where both can reach them. Do not change any behaviour.

Pin a view:

Add a button that pins the current view — the currencies, the date range, and which one gained most — to a table below the chart, so I can compare two different windows side by side.

A second detail:

Add the number of days in the selected window above the chart.

Annotate the chart:

(highlight the chart) Make the lines thicker and put the legend above the chart instead of beside it.

Handle bad input:

If the end date is before the start date, show a clear message instead of an empty chart.